Chương 29

Máy vector hỗ trợ đa lớp

29.1. Giới thiệu

29.1.1. Từ phân loại nhị phân tới phân loại đa lớp

Các mô hình máy vector hỗ trợ đã đề cập (lề cứng, lề mềm, hạt nhân) đều được xây dựng nhằm giải quyết bài toán phân loại nhị phân. Để áp dụng những mô hình này cho bài toán phân loại đa lớp, chúng ta có thể sử dụng các kỹ thuật one-vs-rest hoặc one-vs-one. Cách làm này có những hạn chế như đã trình bày trong Chương 14.

Hồi quy softmax (xem Chương 15) – mô hình tổng quát của hồi quy logistic – được sử dụng phổ biến nhất trong các mô hình phân loại hiện nay. Hồi quy softmax tìm ma trận trọng số $\mathbf{W} \in \mathbb{R}^{d \times C}$ và vector điều chỉnh $\mathbf{b} \in \mathbb{R}^C$ sao cho với mỗi cặp dữ liệu huấn luyện $(\mathbf{x}, y)$, thành phần lớn nhất của vector $\mathbf{z} = \mathbf{W}^T\mathbf{x}$ nằm tại vị trí tương ứng với nhãn y ($y \in \{0, 1, \ldots, C-1\}$). Vector $\mathbf{z}$, còn được gọi là *vector điểm số* (score vector). Để tìm xác suất mỗi điểm dữ liệu rơi vào từng lớp, vector điểm số được đưa qua hàm softmax.

Trong chương này, chúng ta sẽ thảo luận một mô hình khác cũng được áp dụng cho các bài toán phân loại đa lớp – mô hình *SVM đa lớp* (multi-class SVM). Trong đó, ma trận trọng số $\mathbf{W}$ và vector điều chỉnh $\mathbf{b}$ cần được tìm sao cho thành phần cao nhất của vector điểm số nằm tại vị trí ứng với nhãn của dữ liệu đầu vào. Tuy nhiên, hàm mất mát được xây dựng dựa trên ý tưởng của hàm mất mát bản lề thay vì entropy chéo. Hàm mất mát này cũng được tối ưu bởi gradient descent. SVM đa lớp cũng có thể thay thế tầng softmax trong các mạng neuron sâu để tạo ra các bộ phân loại khá hiệu quả.

Hình 29.1. Ví dụ về các bức ảnh trong 10 lớp của bộ dữ liệu CIFAR10 (xem ảnh màu tại trang 407).

Chúng ta sẽ tìm hiểu SVM đa lớp qua ví dụ về bài toán phân loại các bức ảnh thuộc 10 lớp khác nhau trong bộ cơ sở dữ liệu CIFAR10 (https://goo.gl/9KKbQu).

29.1.2. Bộ cơ sở dữ liệu CIFAR10

Bộ cơ sở dữ liệu CIFAR10 gồm 60000 ảnh có kích thước 32×32 điểm ảnh thuộc 10 lớp dữ liệu: *plane, car, bird, cat, deer, dog, frog, horse, ship, và truck*. Một vài ví dụ của mỗi lớp được hiển thị trong Hình 29.1. Tập huấn luyện gồm 50000 bức ảnh, tập kiểm tra gồm 10000 ảnh còn lại. Trong số 50000 ảnh huấn luyện, 1000 ảnh sẽ được lấy ra ngẫu nhiên làm tập xác thực. Đây là một bộ cơ sở dữ liệu tương đối khó vì các bức ảnh có độ phân giải thấp và các đối tượng trong cùng một lớp biến đổi rất nhiều về màu sắc và hình dáng. Thuật toán tốt nhất hiện nay cho bài toán này đã đạt được độ chính xác trên 96% (https://goo.gl/w1sgK4), sử dụng một mạng neuron tích chập đa tầng kết hợp với một hồi quy softmax ở tầng cuối cùng. Trong chương này, chúng ta sẽ sử dụng một mạng neuron đơn giản với một tầng SVM đa lớp để giải quyết bài toán. Mô hình này chỉ mang lại độ chính xác khoảng 40%, nhưng cũng đã rất ấn tượng. Chúng ta sẽ

phân tích mô hình và lập trình chỉ sử dụng thư viện numpy. Bài toán này cũng như nội dung chính của chương được lấy từ ghi chép bài giảng *Linear Classifier II – CS231n 2016* (https://goo.gl/y3QsDP) và *Assignment #1 – CS231n 2016* (https://goo.gl/1Qh84b).

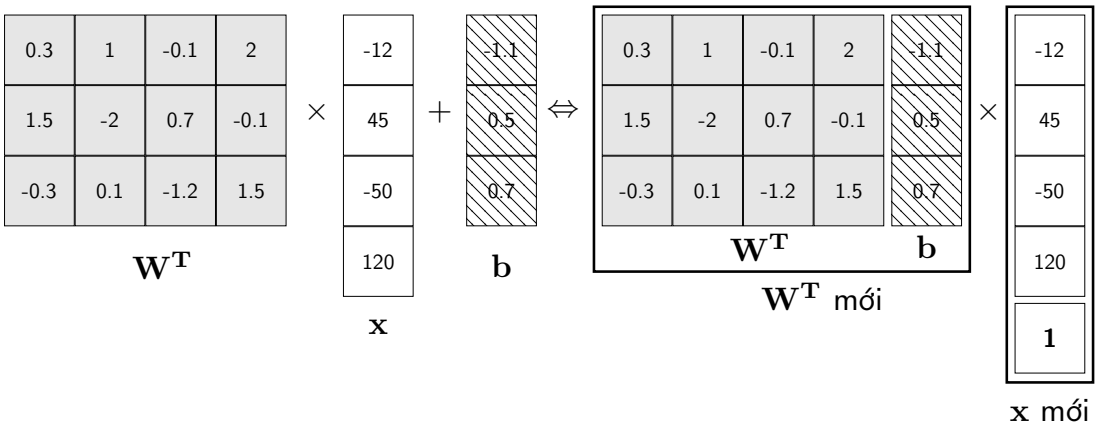
Trước khi đi vào mục xây dựng và tối ưu hàm mất mát cho SVM đa lớp, chúng ta cần xây dựng một bộ trích chọn đặc trưng cho mỗi ảnh.

29.1.3. Xây dựng vector đặc trưng

Sử dung phương pháp xây dựng vector đặc trưng đơn giản nhất: lấy trực tiếp tất cả các điểm trong mỗi ảnh và chuẩn hóa dữ liệu.

- Mỗi ảnh màu của CIFAR-10 có kích thước đều là 32×32 điểm ảnh, vì vậy việc đầu tiên chúng ta có thể làm là kéo dài cả ba kênh *red, green, blue* của bức ảnh thành một vector có kích thước $3 \times 32 \times 32 = 3072$.
- Phương pháp chuẩn hóa dữ liệu đơn giản là trừ mỗi vector đặc trưng đi vector trung bình của dữ liệu trong tập huấn luyện. Việc này sẽ giúp tất cả các thành phần đặc trưng có trung bình bằng không trên tập huấn luyện.

29.1.4. Thủ thuật gộp hệ số điều chỉnh



Hình 29.2. Thủ thuật gộp hệ số điều chỉnh

Với một ma trận trọng số $\mathbf{W} \in \mathbb{R}^{d \times C}$ và vector điều chỉnh $\mathbf{b} \in \mathbb{R}^C$, vector điểm số ứng với một vector đầu vào $\mathbf{x}$ được tính bởi:

$$\mathbf{z} = f(\mathbf{x}, \mathbf{W}, \mathbf{b}) = \mathbf{W}^T\mathbf{x} + \mathbf{b} \tag{29.1}$$

Để biểu thức này đơn giản hơn, ta có thể thêm một phần tử bằng một vào $\mathbf{x}$ và gộp vector điều chỉnh $\mathbf{b}$ vào ma trận trọng số $\mathbf{W}$ như ví dụ trong Hình 29.2. Kỹ

thuật này được gọi là *thuật gộp hệ số điều chỉnh* (bias trick). Từ đây, khi viết $\mathbf{W}$ và $\mathbf{x}$, ta ngầm hiểu chúng đã được mở rộng như phần bên phải của Hình 29.2.

Tiếp theo, chúng ta viết chương trình lấy dữ liệu từ CIFAR10, chuẩn hoá dữ liệu và thêm phần tử bằng một vào cuối mỗi vector đặc trưng. Đồng thời, 1000 dữ liệu từ tập huấn luyện cũng được tách ra làm tập xác thực:

```
from __future__ import print_function
import numpy as np
# need cs231 folder from https://goo.gl/cgJgcG
from cs231n.data_utils import load_CIFAR10

# Load CIFAR 10 dataset
cifar10_dir = 'cs231n/datasets/cifar-10-batches-py'
X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# Extract a validation from X_train
from sklearn.model_selection import train_test_split
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train,
    test_size= 1000)

# mean image of all training images
img_mean = np.mean(X_train, axis = 0)

def feature_engineering(X):
    X -= img_mean # zero-centered
    N = X.shape[0] # number of data point
    X = X.reshape(N, -1) # vectorization
    return np.concatenate((X, np.ones((N, 1))), axis = 1) # bias trick

X_train = feature_engineering(X_train)
X_val   = feature_engineering(X_val)
X_test  = feature_engineering(X_test)
print('X_train shape = ', X_train.shape)
print('X_val shape   = ', X_val.shape)
print('X_test shape  = ', X_test.shape)
```

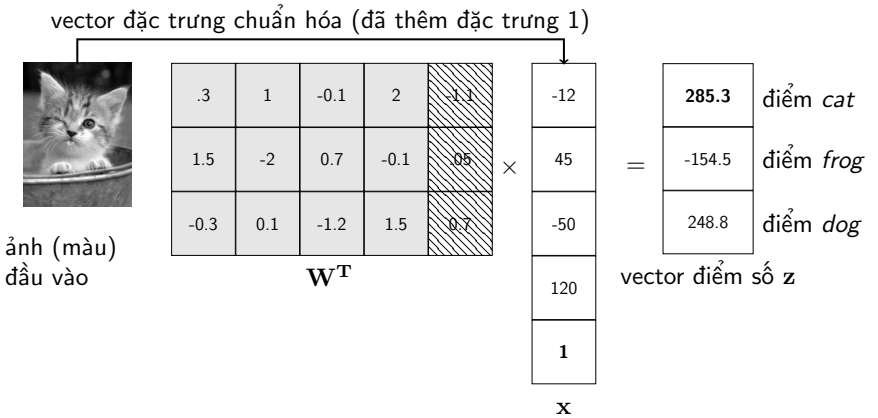
Kết quả:

```
X_train shape =  (49000, 3073)
X_val shape   =  (1000, 3073)
X_test shape  =  (10000, 3073)
```

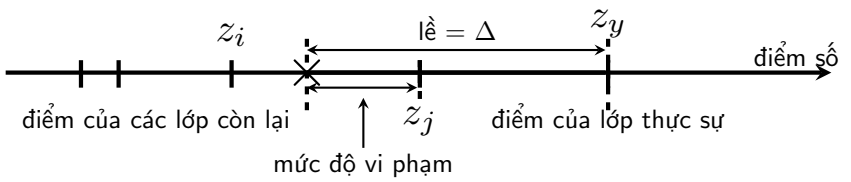
29.2. Xây dựng hàm mất mát

29.2.1. Mất mát bản lề tổng quát cho SVM đa lớp

Trong SVM đa lớp, nhãn của một điểm dữ liệu mới được xác định bởi thành phần có giá trị lớn nhất trong vector điểm số $\mathbf{z} = \mathbf{W}^T\mathbf{x}$ (xem Hình 29.3). Điều này tương tự như hồi quy softmax. Hồi quy softmax sử dụng mất mát entropy chéo để *ép* hai vector xác suất bằng nhau. Việc tối thiểu mất mát entropy chéo



Hình 29.3. Ví dụ về cách tính vector điểm số. Nhãn của một điểm dữ liệu được xác định dựa trên lớp tương ứng có điểm cao nhất.



Hình 29.4. Mô tả mất mát bản lề tổng quát. SVM đa lớp ép điểm số của lớp thực sự (z_y) cao hơn các điểm số khác (z_i) một khoảng cách an toàn Δ. Những điểm số nằm trong vùng an toàn, tức phía trái của điểm $\times$, sẽ gây ra mất mát bằng không. Trong khi đó, những điểm số nằm bên phải điểm $\times$ đã rơi vào vùng không an toàn và cần được gán mất mát dương.

tương đương với việc ép phần tử tương ứng nhãn thực sự trong vector xác suất gần bằng một, đồng thời khiến các phần tử xác suất còn lại gần bằng không. Điều này khiến phần tử tương ứng với nhãn thực sự càng lớn hơn các phần tử còn lại càng tốt. SVM đa lớp sử dụng một giải pháp khác cho mục đích tương tự.

Trong SVM đa lớp, hàm mất mát được xây dựng dựa trên định nghĩa của vùng an toàn giống như SVM lề cứng/mềm cho bài toán phân loại nhị phân. Cụ thể, SVM đa lớp ép thành phần ứng của nhãn thực sự của vector điểm số lớn hơn các phần tử khác; không những thế, nó cần lớn hơn một đại lượng $\Delta > 0$ như được mô tả trong Hình 29.4. Ta gọi đại lượng Δ này là *lề an toàn*.

Nếu điểm số tương ứng với nhãn thực sự lớn hơn các điểm số khác một lượng bằng lề an toàn Δ thì mất mát bằng không. Nói các khác, những điểm số nằm bên trái điểm $\times$ không gây ra mất mát nào. Ngược lại, các điểm số nằm bên phải của $\times$ cần bị *xử phạt*, và mức xử phạt tỉ lệ thuận với độ vi phạm (mức độ vượt quá ranh giới an toàn $\times$).

Để mô tả các mức vi phạm này dưới dạng toán học, trước hết ta giả sử rằng các thành phần của vector điểm số và các lớp dữ liệu được đánh số thứ tự từ một thay vì không như hồi quy softmax. Giả sử rằng điểm dữ liệu $\mathbf{x}$ đang xét có nhãn y và vector điểm số $\mathbf{z} = \mathbf{W}^T\mathbf{x}$. Như vậy, điểm số của nhãn thực sự là z_y, điểm số của các nhãn khác là các $z_i, i \neq y$. Trong Hình 29.4, điểm số z_i nằm trong vùng an toàn còn z_j nằm trong vùng không an toàn. Với mỗi điểm số z_i trong vùng an toàn, mất mát bằng không. Với mỗi điểm số z_j vượt quá $\times$, mất mát được tính bằng khoảng cách từ điểm đó tới $\times$: $z_j - (z_y - \Delta) = \Delta - z_y + z_j$.

Tóm lại, với một điểm số $z_j, j \neq y$, mất mát do nó gây ra là

$$\max(0, \Delta - z_y + z_j) = \max(0, \Delta - \mathbf{w}_y^T\mathbf{x} + \mathbf{w}_j^T\mathbf{x}) \tag{29.2}$$

trong đó $\mathbf{w}_j$ là cột thứ j của ma trận trọng số $\mathbf{W}$. Như vậy, mất mát tại một điểm dữ liệu $\mathbf{x}_n, n = 1, 2, \ldots, N$ vỡi nhãn y_n là

$$\mathcal{L}_n = \sum_{j \neq y_n} \max(0, \Delta - z_{y_n}^n + z_j^n)$$

với $\mathbf{z}^n = \mathbf{W}^T\mathbf{x}_n = [z_1^n, z_2^n, \ldots, z_C^n]^T \in \mathbb{R}^{C\times 1}$ là vector điểm số tương ứng với $\mathbf{x}_n$. Mất mát trên toàn bộ dữ liệu huấn luyện $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N]$, mát mát được định nghĩa là

$$\mathcal{L}(\mathbf{X}, \mathbf{y}, \mathbf{W}) = \frac{1}{N}\sum_{n=1}^{N}\sum_{j \neq y_n} \max(0, \Delta - z_{y_n}^n + z_j^n). \tag{29.3}$$

Trong đó, $\mathbf{y} = [y_1, y_2, \ldots, y_N]$ là vector chứa nhãn thực sự của dữ liệu huấn luyện.

29.2.2. Cơ chế kiểm soát

Điều gì sẽ xảy ra nếu nghiệm tìm được $\mathbf{W}$ là một nghiệm hoàn hảo, tức không có điểm số nào vi phạm và hàm mất mát (29.3) bằng không? Nói cách khác,

$$\Delta - z_{y_n}^n + z_j^n \leq 0 \Leftrightarrow \Delta \leq \mathbf{w}_{y_n}^T\mathbf{x}_n - \mathbf{w}_j^T\mathbf{x}_n \ \forall n = 1, 2, \ldots, N; j = 1, 2, \ldots, C; j \neq y_n$$

Điều này có nghĩa $k\mathbf{W}$ cũng là một nghiệm của bài toán với $k > 1$ bất kỳ. Điều này dẫn tới bài toán có vô số nghiệm và có thể có nghiệm lớn vô cùng. Phương pháp suy giảm trọng số có thể ngăn chặn việc $\mathbf{W}$ trở nên quá lớn:

$$\mathcal{L}(\mathbf{X}, \mathbf{y}, \mathbf{W}) = \underbrace{\frac{1}{N}\sum_{n=1}^{N}\sum_{j \neq y_n} \max(0, \Delta - \mathbf{w}_{y_n}^T\mathbf{x}_n + \mathbf{w}_j^T\mathbf{x}_n)}_{\text{mất mát dữ liệu}} + \underbrace{\frac{\lambda}{2}\|\mathbf{W}\|_F^2}_{\text{suy giảm trọng số}} \tag{29.4}$$

với λ là một tham số kiểm soát dương giúp cân bằng giữa thành phần mất mát dữ liệu và thành phần kiểm soát. Tham số kiểm soát này được chọn bằng xác thực chéo.

29.2.3. Hàm mất mát của SVM đa lớp

Có hai siêu tham số trong hàm mất mát (29.4) là Δ và λ, câu hỏi đặt ra là làm thế nào để chọn ra cặp giá trị hợp lý nhất cho từng bài toán. Liệu chúng ta có cần thực hiện xác thực chéo cho từng giá trị không? Trên thực tế, người ta nhận thấy rằng Δ có thể được chọn bằng một mà không làm ảnh hưởng tới chất lượng của nghiệm (https://goo.gl/NSyfQi). Từ đó, hàm mất mát của SVM có dạng

$$\mathcal{L}(\mathbf{X}, \mathbf{y}, \mathbf{W}) = \frac{1}{N}\sum_{n=1}^{N}\sum_{j\neq y_n}\max(0, 1 - \mathbf{w}_{y_n}^T\mathbf{x}_n + \mathbf{w}_j^T\mathbf{x}_n) + \frac{\lambda}{2}\|\mathbf{W}\|_F^2 \tag{29.5}$$

Nghiệm của bài toán tối thiểu hàm mất mát có thể tìm được bằng gradient descent. Điều này sẽ được thảo luận kỹ trong Mục 29.3.

29.2.4. SVM lề mềm là một trường hợp đặc biệt của SVM đa lớp

(*Hồi quy logistic là một trường hợp đặc biệt của hồi quy softmax.*)

Khi số lớp dữ liệu $C = 2$, tạm bỏ qua mất mát kiểm soát, hàm mất mát tại mỗi điểm dữ liệu trở thành

$$\mathcal{L}_n = \sum_{j\neq y_n}\max(0, 1 - \mathbf{w}_{y_n}^T\mathbf{x}_n + \mathbf{w}_j^T\mathbf{x}_n) \tag{29.6}$$

Xét hai trường hợp:

- $y_n = 1 \Rightarrow \mathcal{L}_n = \max(0, 1 - \mathbf{w}_1^T\mathbf{x}_n + \mathbf{w}_2^T\mathbf{x}_n) = \max(0, 1 - (1)(\mathbf{w}_1 - \mathbf{w}_2)^T\mathbf{x})$
- $y_n = 2 \Rightarrow \mathcal{L}_n = \max(0, 1 - \mathbf{w}_2^T\mathbf{x}_n + \mathbf{w}_1^T\mathbf{x}_n) = \max(0, 1 - (-1)(\mathbf{w}_1 - \mathbf{w}_2)^T\mathbf{x})$

Nếu ta thay $y_n = -1$ cho dữ liệu thuộc lớp có nhãn bằng 2 và đặt $\bar{\mathbf{w}} = \mathbf{w}_1 - \mathbf{w}_2$, hai trường hợp trên có thể được viết gọn thành

$$\mathcal{L}_n = \max(0, 1 - y_n\bar{\mathbf{w}}^T\mathbf{x}_n)$$

Đây chính là mất mát bản lề trong SVM lề mềm. Như vậy, SVM lề mềm là trường hợp đặc biệt của SVM đa lớp khi số lớp dữ liệu $C = 2$.

29.3. Tính toán giá trị và gradient của hàm mất mát

Với những hàm số phức tạp, việc tính toán gradient rất dễ gây ra kết quả không chính xác. Trước khi thực hiện các thuật toán tối ưu sử dụng gradient, ta cần đảm bảo sự chính xác của việc tính gradient. Một lần nữa, có thể sử dụng phương pháp xấp xỉ gradient theo định nghĩa. Để thực hiện phương pháp này, chúng ta cần tính giá trị của hàm mất mát tại một điểm $\mathbf{W}$ bất kỳ.

Việc tính toán giá trị của hàm mất mát và gradient của nó tại **W** bất kỳ không những cần sự chính xác mà còn cần được thực hiện một cách hiệu quả. Để đạt được điều đó, chúng ta sẽ thực hiện từng bước một. Bước thứ nhất phải đảm bảo rằng các tính toán là chính xác, dù cách tính có thể rất chậm. Bước thứ hai là đảm bảo các phép tính được thực hiện một cách hiệu quả. Hai bước này nên được thực hiện trên một lượng dữ liệu nhỏ để có thể nhanh chóng có kết quả. Việc tính xấp xỉ gradient trên dữ liệu lớn thường tốn rất nhiều thời gian vì phải tính giá trị của hàm số trên từng thành phần của ma trận trọng số **W**. Các quy tắc này cũng được áp dụng với những bài toán tối ưu khác có sử dụng gradient trong quá trình tìm nghiệm. Hai mục tiếp theo sẽ mô tả hai bước đã nêu ở trên.

29.3.1. Tính chính xác

Dưới đây là cách tính hàm mất mát và gradient trong (29.5) bằng hai vòng **for**:

```
def svm_loss_naive(W, X, y, reg):
    ''' calculate loss and gradient of the loss function at W. Naive way
    W: 2d numpy array of shape (d, C). The weight matrix.
    X: 2d numpy array of shape (N, d). The training data
    y: 1d numpy array of shape (N,). The training label
    reg: a positive number. The regularization parameter
    '''
    d, C = W.shape # data dim, No. classes
    N = X.shape[0] # No. points
    loss = 0
    dW = np.zeros_like(W)
    for n in xrange(N):
        xn = X[n]
        score = xn.dot(W)
        for j in xrange(C):
            if j == y[n]:
                continue
            margin = 1 - score[y[n]] + score[j]
            if margin > 0:
                loss += margin
                dW[:, j] += xn
                dW[:, y[n]] -= xn

    loss /= N
    loss += 0.5*reg*np.sum(W * W) # regularization
    dW /= N
    dW += reg*W
    return loss, dW

# random, small data
d, C, N = 100, 3, 300
reg = .1
W_rand = np.random.randn(d, C)
X_rand = np.random.randn(N, d)
y_rand = np.random.randint(0, C, N)
# sanity check
print('Loss with reg = 0  :', svm_loss_naive(W_rand, X_rand, y_rand, 0)[0])
print('Loss with reg = 0.1:', svm_loss_naive(W_rand, X_rand, y_rand, .1)[0])
```

Kết quả:

```
Loss with reg = 0  : 12.5026818221
Loss with reg = 0.1: 27.7805360552
```

Cách tính với hai vòng **for** lồng nhau như trên mô tả chính xác biểu thức (29.5) nên tính chính xác có thể được đảm bảo. Việc kiểm tra ở cuối cho cái nhìn ban đầu về hàm mất mát: dương và không có kiểm soát sẽ cho giá trị cao hơn.

Cách tính gradient cho phần mất mát dữ liệu dựa trên nhận xét sau đây:

$$\nabla_{\mathbf{w}_{y_n}} \max(0, 1 - \mathbf{w}_{y_n}^T\mathbf{x}_n + \mathbf{w}_j^T\mathbf{x}_n) = \begin{cases} 0 & \text{nếu } 1 - \mathbf{w}_{y_n}^T\mathbf{x}_n + \mathbf{w}_j^T\mathbf{x}_n < 0 \\ -\mathbf{x}_n & \text{nếu } 1 - \mathbf{w}_{y_n}^T\mathbf{x}_n + \mathbf{w}_j^T\mathbf{x}_n > 0 \end{cases} \tag{29.7}$$

$$\nabla_{\mathbf{w}_j} \max(0, 1 - \mathbf{w}_j^T\mathbf{x}_n + \mathbf{w}_j^T\mathbf{x}_n) = \begin{cases} 0 & \text{nếu } 1 - \mathbf{w}_{y_n}^T\mathbf{x}_n + \mathbf{w}_j^T\mathbf{x}_n < 0 \\ \mathbf{x}_n & \text{nếu } 1 - \mathbf{w}_{y_n}^T\mathbf{x}_n + \mathbf{w}_j^T\mathbf{x}_n > 0 \end{cases} \tag{29.8}$$

Mặc dù gradient không xác định tại các điểm mà $1 - \mathbf{w}_{y_n}^T\mathbf{x}_n + \mathbf{w}_j^T\mathbf{x}_n = 0$, ta vẫn có thể giả sử rằng gradient tại 0 cũng bằng 0.

Việc kiểm tra tính chính xác của gradient bằng phương pháp xấp xỉ gradient theo định nghĩa xin dành lại cho bạn đọc.

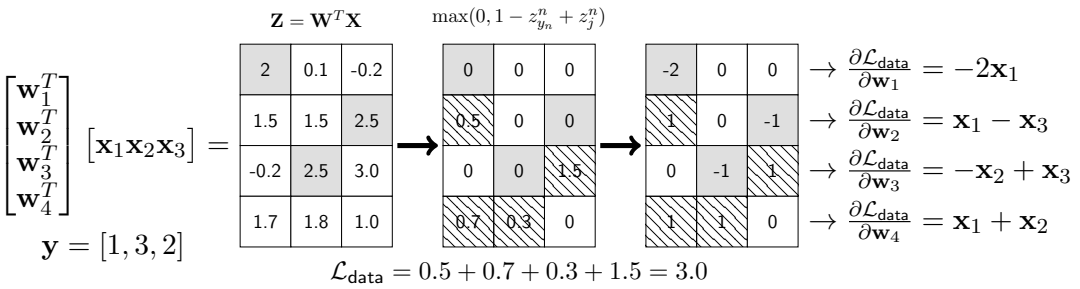
Khi tính chính xác của gradient đã được đảm bảo, ta cần một cách hiệu quả để tính gradient.

29.3.2. Tính hiệu quả

Cách tính toán hiệu quả thường không chứa các vòng **for** mà được viết gọn lại sử dụng các kỹ thuật vector hóa. Để dễ hình dung, chúng ta cùng quan sát Hình 29.5. Ở đây, chúng ta tạm quên thành phần kiểm soát vì giá trị và gradient của thành phần này đều được tính một cách đơn giản. Chúng ta cũng bỏ qua hệ số $\frac{1}{N}$ cho các phép tính đơn giản hơn.

Giả sử có bốn lớp dữ liệu và mini-batch $\mathbf{X}$ gồm ba điểm dữ liệu $\mathbf{X} = \begin{bmatrix}\mathbf{x}_1 & \mathbf{x}_2 & \mathbf{x}_3\end{bmatrix}$ lần lượt thuộc các lớp 1, 3, 2 (vector $\mathbf{y}$). Các ô có nền màu xám ở mỗi cột tương ứng với nhãn thực sự của điểm dữ liệu. Các bước tính giá trị và gradient của hàm mất mát có thể được hình dung như sau:

- Bước 1: Tính ma trận điểm số $\mathbf{Z} = \mathbf{W}^T\mathbf{X}$.
- Bước 2: Với mỗi ô, tính $\max(0, 1 - \mathbf{w}_{y_n}^T\mathbf{x}_n + \mathbf{w}_j^T\mathbf{x}_n)$. Vì biểu thức hàm mất mát cho một điểm dữ liệu không chứa thành phần $j = y_n$ nên ta không cần tính các ô có nền màu xám. Sau khi tính được giá trị của từng ô, ta chỉ quan tâm tới các ô có giá trị lớn hơn 0 – các ô có nền sọc chéo. Lấy tổng tất cả



Hình 29.5. Mô phỏng cách tính hàm mất mát và gradient trong SVM đa lớp.

các phần tử của các ô nền sọc chéo, ta được giá trị của hàm mất mát. Ví dụ, nhìn vào ma trận ở giữa trong Hình 29.5, giá trị hàng thứ hai, cột thứ nhất bằng $\max(0, 1 - 2 + 1.5) = \max(0, 0.5) = 0.5$. Giá trị hàng thứ ba, cột thứ nhất bằng $\max(0, 1 - 2 + (-0.2)) = \max(0, -1.2) = 0$. Giá trị hàng thứ tư, cột thứ nhất bằng $\max(0, 1 - 2 + 1.7) = 0.7$. Tương tự với các cột còn lại.

- Bước 3: Theo công thức (29.7) và (29.8), với ô nền sọc ở hàng thứ hai, cột thứ nhất (ứng với điểm dữ liệu $\mathbf{x}_1$), gradient theo vector trọng số $\mathbf{w}_2$ được cộng thêm một lượng $\mathbf{x}_1$ và gradient theo vector trọng số $\mathbf{w}_1$ bị trừ đi một lượng $\mathbf{x}_1$. Như vậy, trong cột thứ nhất, có bao nhiêu ô nền sọc thì có bấy nhiêu lần gradient của $\mathbf{w}_1$ bị trừ đi một lượng $\mathbf{x}_1$. Xét ma trận bên phải, giá trị của ô ở hàng thứ i, cột thứ j là hệ số của gradient theo $\mathbf{w}_i$ gây ra bởi điểm dữ liệu $\mathbf{x}_j$. Tất cả các ô nền sọc đều có giá trị bằng 1. Ô màu xám ở cột thứ nhất phải bằng -2 vì cột đó có hai ô nền sọc. Tương tự với các ô nền sọc và xám còn lại.

- Bước 4: Cộng theo mỗi hàng, ta được gradient theo hệ số của lớp tương ứng.

Cách tính toán trên đây có thể thực hiện như sau:

```
def svm_loss_vectorized(W, X, y, reg):
    d, C = W.shape
    N = X.shape[0]
    loss = 0
    dW = np.zeros_like(W)
    Z = X.dot(W) # shape (N, C)
    id0 = np.arange(Z.shape[0])
    correct_class_score = Z[id0, y].reshape(N, 1) # shape (N, 1)
    margins = np.maximum(0, Z - correct_class_score + 1) # shape (N, C)
    margins[id0, y] = 0
    loss = np.sum(margins)
    loss /= N
    loss += 0.5 * reg * np.sum(W * W)

    F = (margins > 0).astype(int)# shape (N, C)
    F[np.arange(F.shape[0]), y] = np.sum(-F, axis = 1)
    dW = X.T.dot(F)/N + reg*W
    return loss, dW
```

Đoạn mã phía trên không chứa vòng **for** nào. Để kiểm tra tính chính xác và hiệu quả của hàm này, chúng ta cần kiểm chứng ba điều. (i) Giá trị hàm mất mát đã chính xác? (ii) Giá trị gradient đã chính xác? (iii) Cách tính đã thực sự hiệu quả?:

```
d, C = 3073, 10
W_rand = np.random.randn(d, C)
import time
t1 = time.time()
l1, dW1 = svm_loss_naive(W_rand, X_train, y_train, reg)
t2 = time.time()
l2, dW2 = svm_loss_vectorized(W_rand, X_train, y_train, reg)
t3 = time.time()
print('Naive       --  run time:', t2 - t1, '(s)')
print('Vectorized  --  run time:', t3 - t2, '(s)')
print('loss difference:', np.linalg.norm(l1 - l2))
print('gradient difference:', np.linalg.norm(dW1 - dW2))
```

Kết quả:

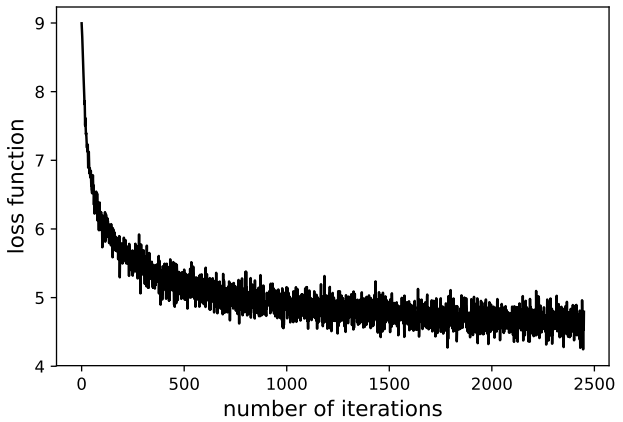
```
Naive       --  run time: 7.34640693665 (s)
Vectorized  --  run time: 0.365024089813 (s)
loss difference: 8.73114913702e-11
gradient difference: 1.87942037251e-10
```

Kết quả cho thấy cách tính vector hóa hiệu quả hơn khoảng 20 lần và sự chênh lệch giữa hai cách tính là không đáng kể. Như vậy cả tính chính xác và tính hiệu quả đều đã được đảm bảo.

29.3.3. Mini-batch gradient descent cho SVM đa lớp

Việc huấn luyện SVM đa lớp có thể thực hiện như sau:

```
def multiclass_svm_GD(X, y, Winit, reg, lr=.1,
                      batch_size = 1000, num_iters = 50, print_every = 10):
    W = Winit
    loss_history = []
    for it in xrange(num_iters):
        mix_ids = np.random.permutation(X.shape[0])
        n_batches = int(np.ceil(X.shape[0]/float(batch_size)))
        for ib in range(n_batches):
            ids = mix_ids[batch_size*ib: min(batch_size*(ib+1), X.shape[0])]
            X_batch = X[ids]
            y_batch = y[ids]
            lossib, dW = svm_loss_vectorized(W, X_batch, y_batch, reg)
            loss_history.append(lossib)
            W -= lr*dW
        if it % print_every == 0 and it > 0:
            print('it %d/%d, loss = %f' %(it, num_iters, loss_history[it]))
    return W, loss_history
```



Hình 29.6. Lịch sử giá trị hàm mất mát qua các vòng lặp. Ta thấy rằng mất mát có xu hướng giảm và hội tụ khá nhanh.

```
d, C = X_train.shape[1], 10
reg = .1
W = 0.00001*np.random.randn(d, C)

W, loss_history = multiclass_svm_GD(X_train, y_train, W, reg, lr = 1e-8,
    num_iters = 50, print_every = 5)
```

Kết quả:

```
epoch 5/50, loss = 5.482782
epoch 10/50, loss = 5.204365
epoch 15/50, loss = 4.885159
epoch 20/50, loss = 5.051539
epoch 25/50, loss = 5.060423
epoch 30/50, loss = 4.691241
epoch 35/50, loss = 4.841132
epoch 40/50, loss = 4.643097
epoch 45/50, loss = 4.691177
```

Ta thấy rằng giá trị hàm mất mát có xu hướng giảm và hội tụ nhanh. Giá trị hàm mất mát sau mỗi vòng lặp được minh hoạ trong Hình 29.6.

Sau khi đã tìm được ma trận trọng số **W**, chúng ta cần viết các hàm xác định nhãn của các điểm dữ liệu mới và đánh giá độ chính xác của mô hình:

```
def multisvm_predict(W, X):
    Z = X.dot(W)
    return np.argmax(Z, axis=1)

def evaluate(W, X, y):
    y_pred = multisvm_predict(W, X)
    acc = 100*np.mean(y_pred == y)
    return acc
```

Tiếp theo, ta sử dụng tập xác thực để chọn ra bộ siêu tham số mô hình phù hợp. Có hai siêu tham số trong thuật toán tối ưu SVM đa lớp: tham số kiểm soát và tốc độ học. Hai tham số này sẽ được tìm bằng phương pháp *tìm trên lưới* (grid search). Bộ giá trị mang lại độ chính xác trên tập xác thực cao nhất sẽ được dùng để đánh giá tập kiểm tra:

```
lrs = [1e-9, 1e-8, 1e-7, 1e-6]
regs = [0.1, 0.01, 0.001, 0.0001]
best_W = 0
best_acc = 0
for lr in lrs:
    for reg in regs:
        W, loss_history = multiclass_svm_GD(X_train, y_train, W, reg, \
                lr = 1e-8, num_iters = 100, print_every = 1e20)
        acc = evaluate(W, X_val, y_val)
        print('lr = %e, reg = %e, loss = %f, val acc = %.2f'
              %(lr, reg, loss_history[-1], acc))
        if acc > best_acc:
            best_acc, best_W = acc, W
```

Kết quả:

```
lr = 1.000000e-09, reg = 1.000000e-01, loss = 4.422479, val acc = 40.30
lr = 1.000000e-09, reg = 1.000000e-02, loss = 4.474095, val acc = 40.70
lr = 1.000000e-09, reg = 1.000000e-03, loss = 4.240144, val acc = 40.90
lr = 1.000000e-09, reg = 1.000000e-04, loss = 4.257436, val acc = 41.40
lr = 1.000000e-08, reg = 1.000000e-01, loss = 4.482856, val acc = 41.50
lr = 1.000000e-08, reg = 1.000000e-02, loss = 4.036566, val acc = 41.40
lr = 1.000000e-08, reg = 1.000000e-03, loss = 4.085053, val acc = 41.00
lr = 1.000000e-08, reg = 1.000000e-04, loss = 3.891934, val acc = 41.40
lr = 1.000000e-07, reg = 1.000000e-01, loss = 3.947408, val acc = 41.50
lr = 1.000000e-07, reg = 1.000000e-02, loss = 4.088984, val acc = 41.90
lr = 1.000000e-07, reg = 1.000000e-03, loss = 4.073365, val acc = 41.70
lr = 1.000000e-07, reg = 1.000000e-04, loss = 4.006863, val acc = 41.80
lr = 1.000000e-06, reg = 1.000000e-01, loss = 3.851727, val acc = 41.90
lr = 1.000000e-06, reg = 1.000000e-02, loss = 3.941015, val acc = 41.80
lr = 1.000000e-06, reg = 1.000000e-03, loss = 3.995598, val acc = 41.60
lr = 1.000000e-06, reg = 1.000000e-04, loss = 3.857822, val acc = 41.80
```

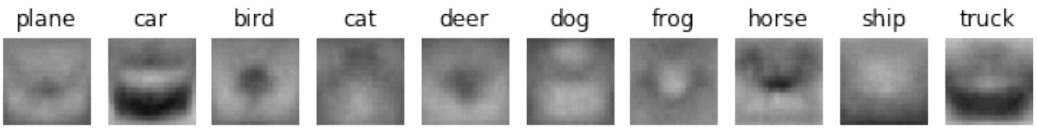
Như vậy, độ chính xác cao nhất cho tập xác thực là 41.9%. Ma trận trọng số $\mathbf{W}$ tốt nhất đã được lưu trong biến `best_W`. Áp dụng mô hình này lên tập kiểm tra:

```
acc = evaluate(best_W, X_test, y_test)
print('Accuracy on test data = %2f %%'%acc)
```

Kết quả:

```
Accuracy on test data =  39.88 %
```

Như vậy, kết quả đạt được rơi vào khoảng gần 40 %.



Hình 29.7. Minh họa hệ số tìm được dưới dạng các bức ảnh (xem ảnh màu tại trang 407).

29.3.4. Minh họa nghiệm tìm được

Để ý rằng mỗi $\mathbf{w}_i$ có chiều bằng chiều của dữ liệu. Bằng cách loại bỏ các hệ số điều chỉnh ở cuối và sắp xếp lại các điểm ảnh của mỗi trong 10 vector trọng số tìm được, ta sẽ thu được các bức ảnh có cùng kích thước $3 \times 32 \times 32$ như mỗi ảnh nhỏ trong cơ sở dữ liệu. Hình 29.7 minh họa các vector trọng số $\mathbf{w}_i$ tìm được.

Ta thấy rằng vector trọng số tương ứng với mỗi lớp khá giống các bức ảnh trong lớp đó, ví dụ *car* và *truck*. Vector trọng số của *ship* và *plane* có mang màu xanh của nước biển và bầu trời (xem ảnh màu tại trang 407). Trong khi đó, *horse* giống như một con ngựa hai đầu; điều này dễ hiểu vì các con ngựa có thể quay đầu về hai phía trong tập huấn luyện. Có thể nói rằng các hệ số tìm được là ảnh đại diện của mỗi lớp.

Xin nhắc lại, nhãn của mỗi điểm dữ liệu được xác định bởi vị trí của thành phần có giá trị cao nhất trong vector điểm số $\mathbf{z} = \mathbf{W}^T\mathbf{x}$:

$$\text{class}(\mathbf{x}) = \arg\max_{i=1,2,\ldots,C} \mathbf{w}_i^T\mathbf{x}$$

Để ý rằng tích vô hướng chính là đại lượng đo sự tương quan giữa hai vector. Đại lượng này càng lớn thì sự tương quan càng cao, tức hai vector càng giống nhau. Như vậy, việc đi tìm nhãn của một bức ảnh mới chính là việc đi tìm bức ảnh đó giống với bức ảnh đại diện cho lớp nào nhất. Kỹ thuật này này khá giống với KNN, nhưng chỉ có 10 tích vô hướng cần được tính thay vì khoảng cách tới mọi điểm dữ liệu huấn luyện.

29.4. Thảo luận

- Giống như hồi quy softmax, SVM đa lớp vẫn được coi là một bộ phân loại tuyến tính vì đường ranh giới giữa các lớp là các đường tuyến tính.

- SVM hạt nhân hoạt động khá tốt, nhưng việc tính toán ma trận hạt nhân có thể tốn nhiều thời gian và bộ nhớ. Hơn nữa, việc mở rộng SVM hạt nhân cho bài toán phân loại đa lớp thường không hiệu quả bằng SVM đa lớp vì kỹ thuật được sử dụng vẫn là one-vs-rest. Một ưu điểm nữa của SVM đa lớp là nó có thể được tối ưu bằng các phương pháp gradient descent, phù hợp với

các bài toán với dữ liệu lớn. Ngoài ra, SVM đa lớp có thể được kết hợp với các mạng neuron đa tầng trong trường hợp dữ liệu không tách biệt tuyến tính.

- Trên thực tế, SVM đa lớp và hồi quy softmax có hiệu quả tương đương nhau (xem https://goo.gl/xLccj3). Có thể trong một bài toán cụ thể, phương pháp này tốt hơn phương pháp kia nhưng điều ngược lại xảy ra trong các bài toán khác. Khi thực hành, ta có thể thử cả hai phương pháp rồi chọn phương pháp cho kết quả tốt hơn.